MASTER OF SCIENCE
IN ENGINEERING

Word Embeddings

Deep learning with word representations

Jean Hennebert
Martin Melchior

Overview

1. Word embeddings - general concepts
2. Word2Vec - principles and examples
3. Word2Vec - training strategies
4. Other embeddings
   - GloVe
   - FastText
   - ELMo

Practical work

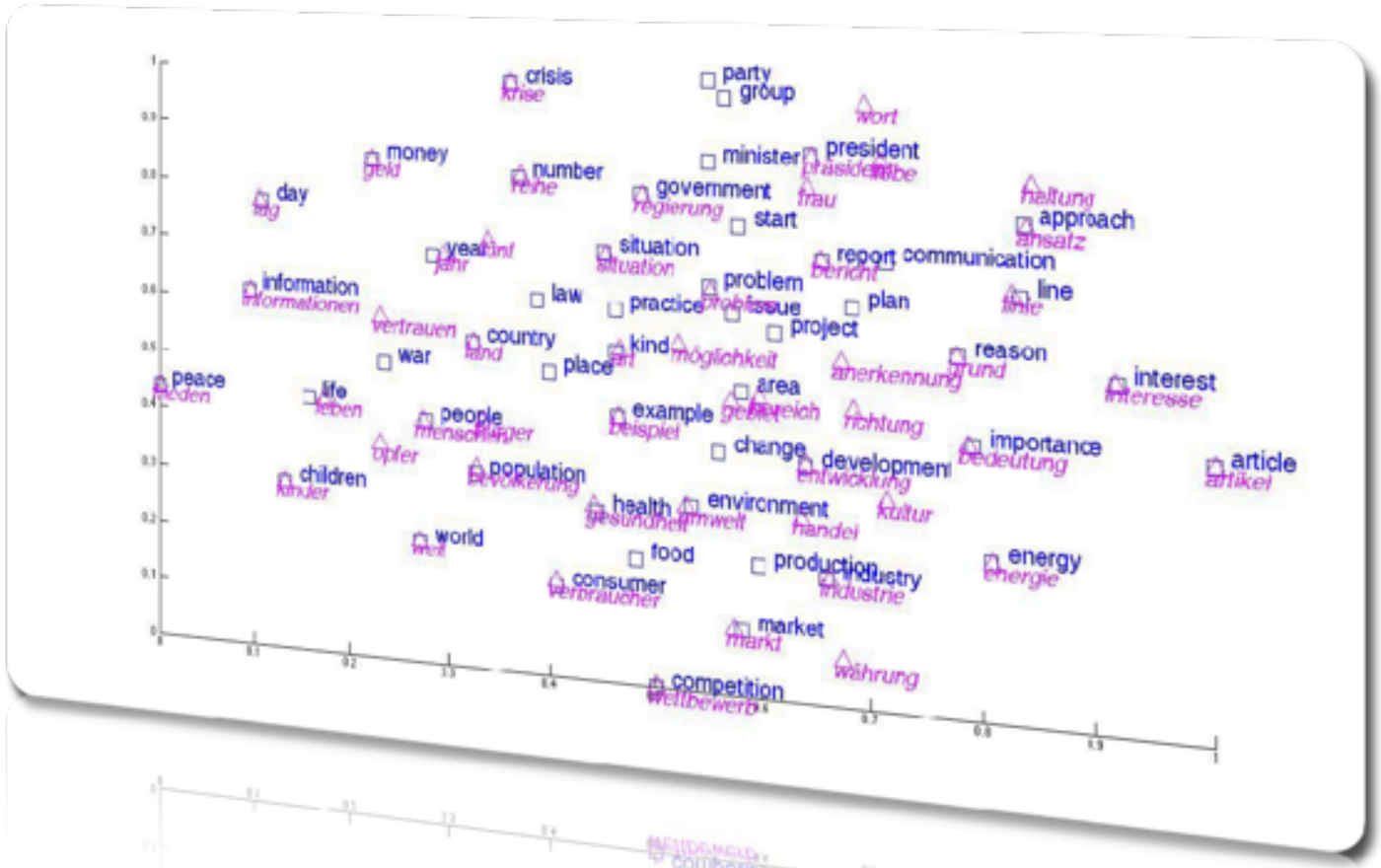
1. Word Embedding

Why embedding

Principles

Embedding layer

Keras implementation



Why word embedding?

- Inputs of networks need to be numerical continuous values
- Using 1-hot “atomic” representation of words to feed the classifier is generally not working!
  - Let’s imagine a 50’000 word vocabulary application with 1-hot representation.

apple	[0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 ... 0 0 0 0 0 0]
orange	[0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 ... 0 0 0 0 0 0]
car	[0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 ... 0 0 0 0 0 0]

  - With 1-hot encoding, information is “too sparse”
    - There is no information apart the presence or not of a word
    - There is no “distribution” of relevant information on different inputs
    - There is no “continuity” of the representation

Figures taken from Xin Ring - School of Information - University of Michigan

Can we think about a better encoding?

Activity

- e.g. associating features to the columns

Apple					
Orange					
Car					
Bike					

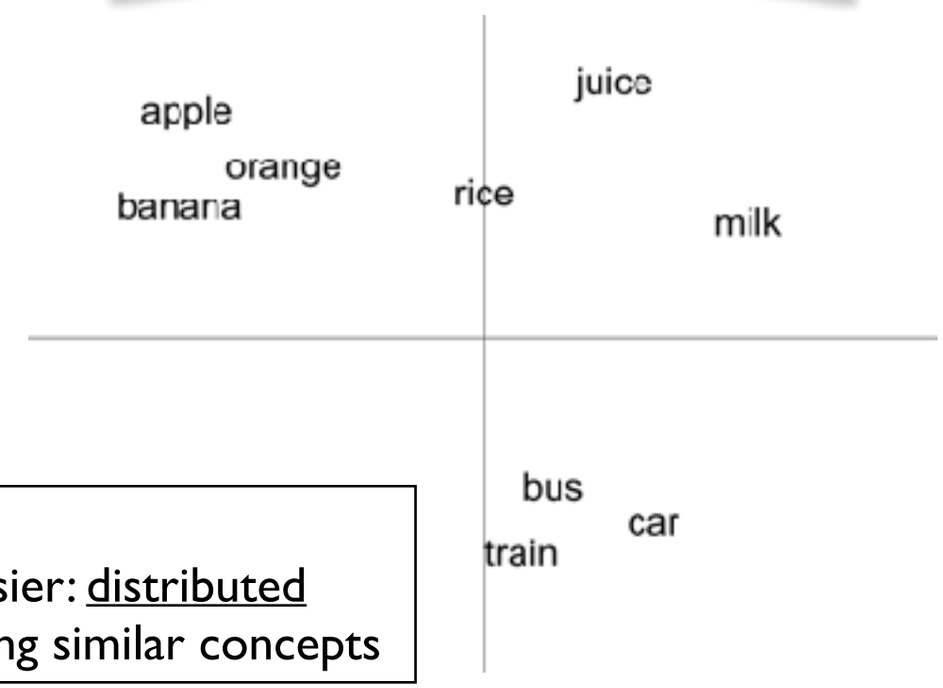
Word embedding

Word embedding is a projection of a word into vectors of real numbers, i.e. a mapping of a space where each word has its own dimension to a space that is of lower dimensionality.

Example: projection into a 3 dimensional space

anarchism	0.5	0.1	−0.1
originated	−0.5	0.3	0.9
as	0.3	−0.5	−0.3
a	0.7	0.2	−0.3
term	0.8	0.1	−0.1
of	0.4	−0.6	−0.1
abuse	0.7	0.1	−0.4

Example: projection into a 2 dimensional space



A "wanted" property of the projection:

1. Expose "good" features, i.e. values that makes the task easier: <u>distributed representation</u> of concepts and <u>continuity</u> of words sharing similar concepts

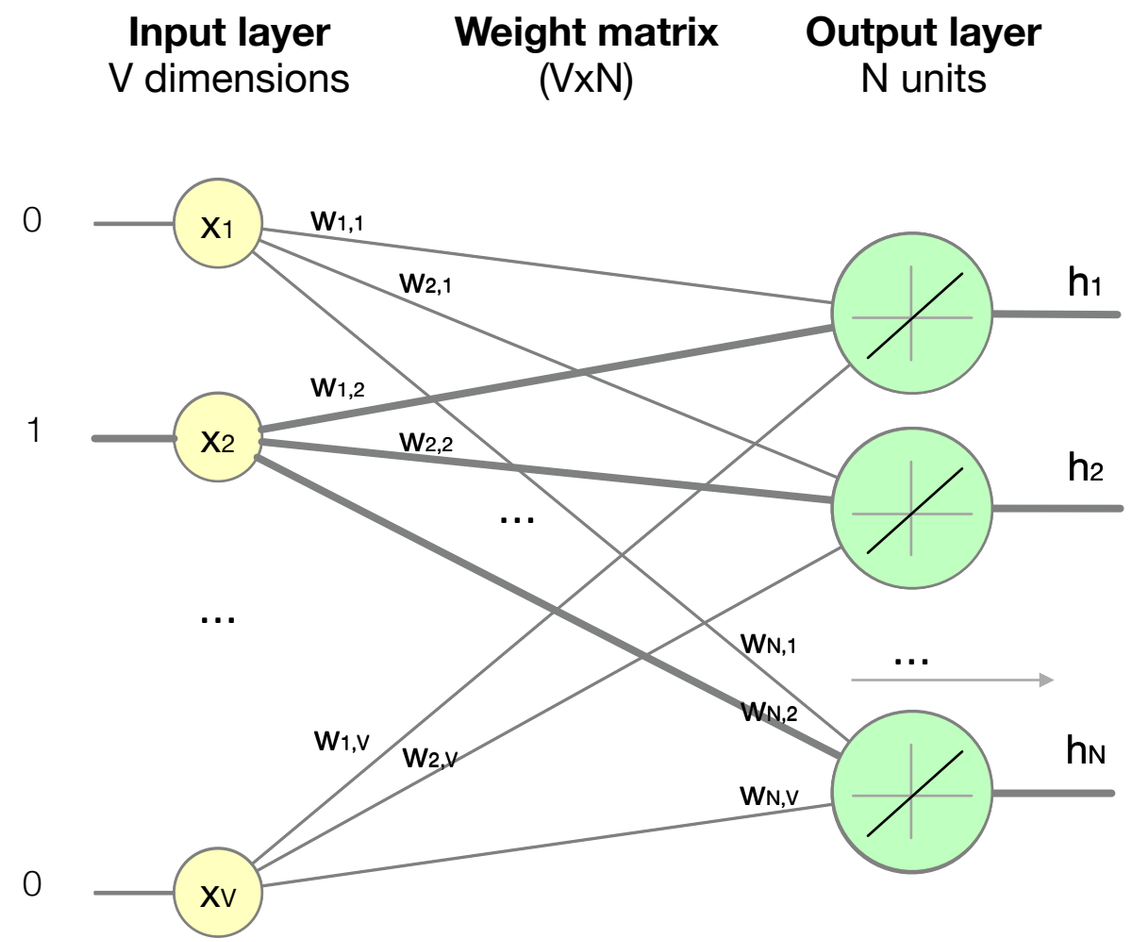
Yes, great, but…

- Hand-crafting such embedding would be a nightmare
  - What are the good and needed column features?
    - Good features for a task A could be bad for a task B
  - With 2 minutes to define features per words, a 50K vocabulary task would need ~1’700 h, i.e. 1 person/year - too long
- Good news: some people invented a way to learn the embedding from data 😅

Embedding layer

- Input: 1-hot encoding of the word in a vocabulary of V dimension
  - Or just the index of the input that is “hot”
- Output: 1 vector of N dimension

Example: ’orange’ is the 2nd word in the vocabulary



Output for ’orange’ is a vector such as [0.3, 0.1, -0.2, …]

$$\mathbf{h} = \mathbf{x}^T \mathbf{W} := \mathbf{v}_{w_I}$$

As ‘orange’ is 1-hot, say in position 2 of vector x, h is then a simple <u>lookup</u> of the values in line 2 of the input weight matrix

Use of embedding layers

- Analogy with CNN for image recognition
  - The embedding layer is the first layer of a NLP network
  - It performs a kind of feature extraction for words
- 3 strategies:
  - Train the embedding layer on a given data set and for a given task
    - The embedding is going to be specific to the dataset and the task
  - Use a pre-trained “generic” embedding such as word2vec or GloVe and freeze the layer - similar to *transfer learning* concept
  - Do both: init the embedding layer with pre-trained weights and fine tune it through training.

Embedding layer in Keras

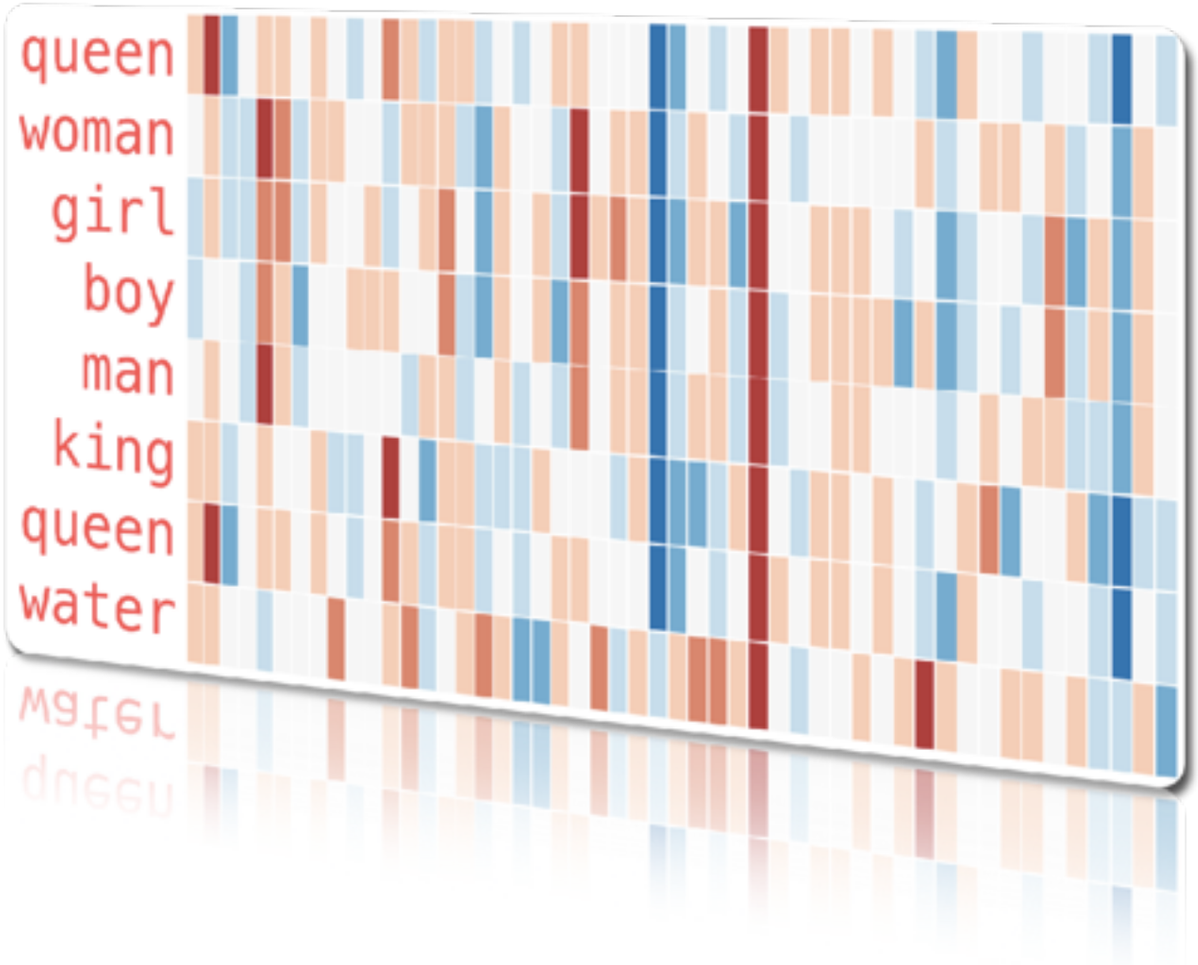
```
# Create the model
top_words = 5000
embedding_vector_length = 32

model = Sequential()
model.add(Embedding(top_words, embedding_vector_length, input_length=max_review_length))
model.add(LSTM(100))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy',optimizer='adam', metrics=['accuracy'])

print(model.summary())
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
embedding_5 (Embedding)      (None, 500, 32)           160000
_________________________________________________________________
lstm_4 (LSTM)                (None, 100)               53200
_________________________________________________________________
dense_5 (Dense)              (None, 1)                 101
=================================================================
Total params: 213,301
Trainable params: 213,301
Non-trainable params: 0
_________________________________________________________________
```

- Used for the IMDB movie sentiment analysis task (see exercise on the use of recurrent networks and LSTM)
- The embedding layer will learn a word embedding for all the words of the dataset. It has 3 arguments:
  - (1) the input dimension, i.e. the size of the vocabulary (in our case the 5000 top words),
  - (2) the output dimension, i.e. the dimension of the vector space in which words will be embedded (in our case, we set it to 32)
  - (3) the input length, i.e. the length of input sequences, namely the reviews (for ex 500 words).
- Pre-trained word embeddings, such as GloVE or Word2Vec, are also available if you don't want to train your embeddings by yourself.

2. Word2Vec

- Principles
- Distributed and contextual representations
- Geometric operations
- Examples



http://jalammar.github.io/illustrated-word2vec/

word2vec - principles

word2vec relates to models producing word embedding into space with **contextual similarity** or words, i.e. words that share common context are located in close proximity.

- Two original papers from Google - Mikolov et al. (2013)

Efficient Estimation of Word Representations in Vector Space

Tomas Mikolov
Google Inc., Mountain View, CA
tmikolov@google.com

Kai Chen
Google Inc., Mountain View, CA
kaichen@google.com

Greg Corrado
Google Inc., Mountain View, CA
gcorrado@google.com

Jeffrey Dean
Google Inc., Mountain View, CA
jeff@google.com

Abstract

We propose two novel model architectures for computing continuous vector representations of words from very large data sets. The quality of these representations is measured in a word similarity task, and the results are compared to the previously best performing techniques based on different types of neural networks. We observe large improvements in accuracy at much lower computational cost, i.e. it takes less than a day to learn high quality word vectors from a 1.6 billion words data set. Furthermore, we show that these vectors provide state-of-the-art performance on our test set for measuring syntactic and semantic word similarities.

https://arxiv.org/pdf/1301.3781.pdf

Distributed Representations of Words and Phrases and their Compositionality

Tomas Mikolov
Google Inc.
Mountain View
mikolov@google.com

Ilya Sutskever
Google Inc.
Mountain View
ilyasu@google.com

Kai Chen
Google Inc.
Mountain View
kai@google.com

Greg Corrado
Google Inc.
Mountain View
gcorrado@google.com

Jeffrey Dean
Google Inc.
Mountain View
jeff@google.com

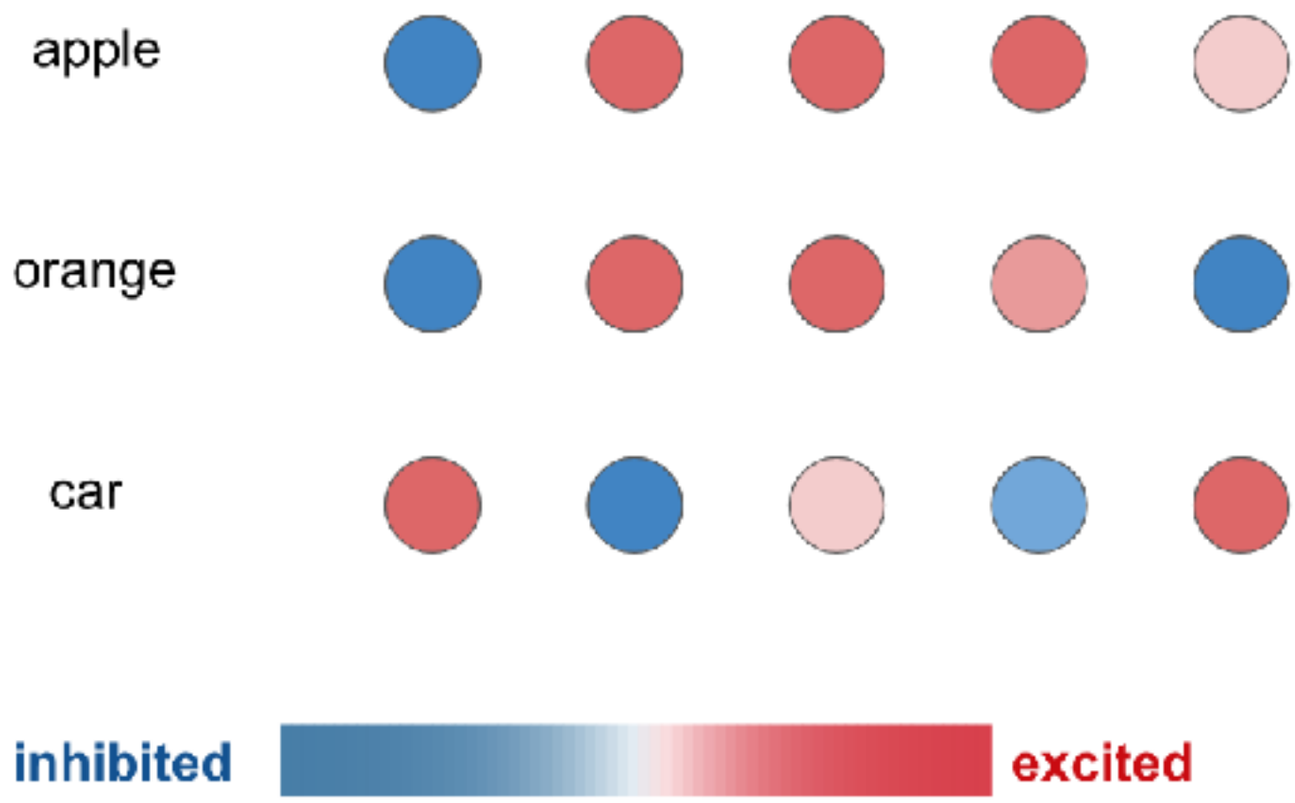
Abstract

The recently introduced continuous Skip-gram model is an efficient method for learning high-quality distributed vector representations that capture a large number of precise syntactic and semantic word relationships. In this paper we present several extensions that improve both the quality of the vectors and the training speed. By subsampling of the frequent words we obtain significant speedup and also learn more regular word representations. We also describe a simple alternative to the hierarchical softmax called negative sampling.

https://arxiv.org/abs/1310.4546

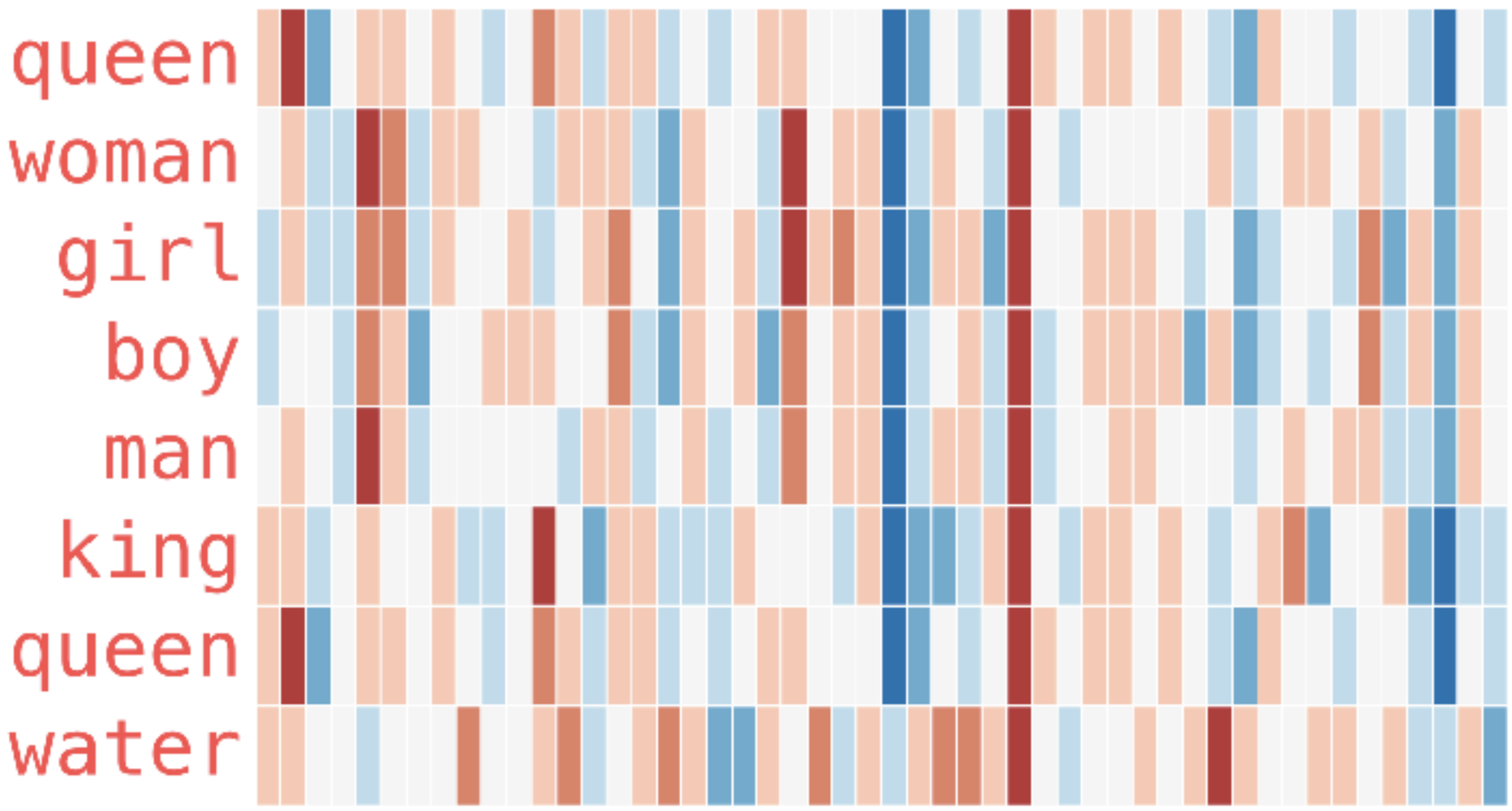
word2vec - distributed representation

- words in similar context share similar activations



Figures taken from Xin Ring - School of Information - University of Michigan

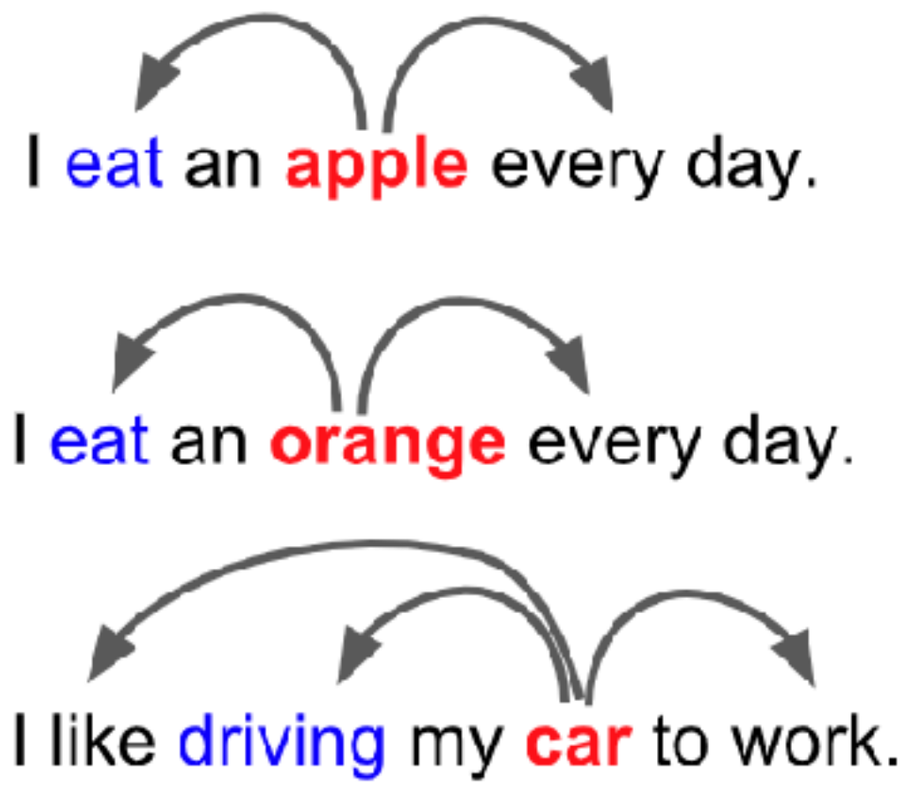
word2vec - distributed representation



- From “http://jalammar.github.io/illustrated-word2vec/”
  - We don’t know what each dimensions codes for.
  - There’s a straight red column through all of these different words. Could it be noun versus verb? Possible
  - You can see how “woman” and “girl” are similar to each other in a lot of places. The same with “man” and “boy”
  - “boy” and “girl” have places where they are similar to each other, but different from “woman” or “man”. Could these be coding for a vague conception of youth? Possible.
  - All but the last word are representing people. You can observe the blue column going all the way down and stopping before the embedding for “water”. Could this column be associated to “human”. Possible.
  - There are clear places where “king” and “queen” are similar to each other and distinct from all the others. Could these be coding for a vague concept of royalty? Possible.

word2vec - contextual representation

- Hypothesis: a word is represented by its “context”



Another hypothesis: the order of the context words is disregarded. This is the “**bag-of-word**” assumption.

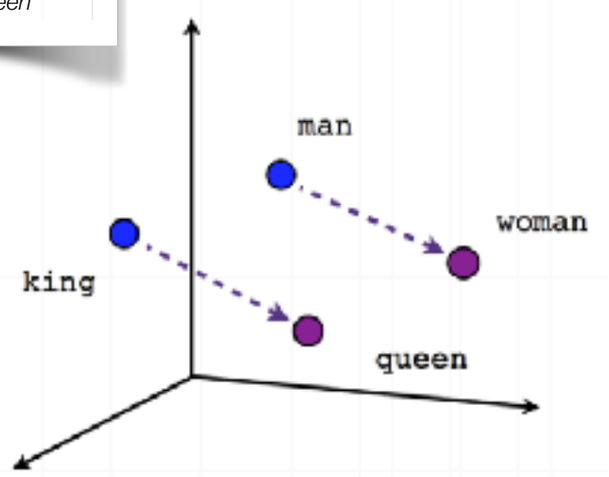
A window around the target word is created and words falling in the window are added to the “bag”, disregarding the order.

Figures taken from Xin Ring - School of Information - University of Michigan

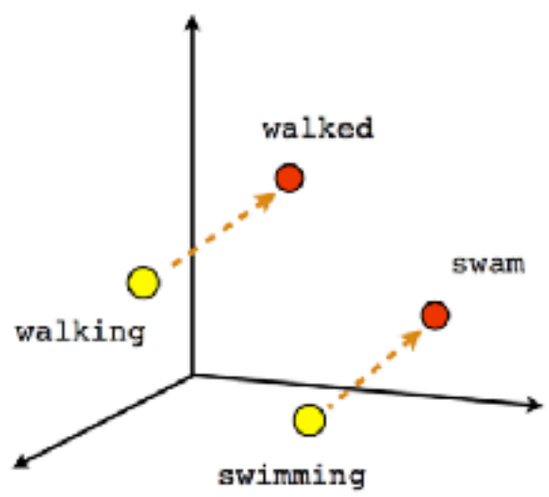
word2vec - geometric operations

- The word2vec mapping is not only about clustering related words, it is also able to preserve some **“path meaning”**

In vector terms:
$v_{woman} + (v_{king} - v_{man})$
is close to v_{queen}

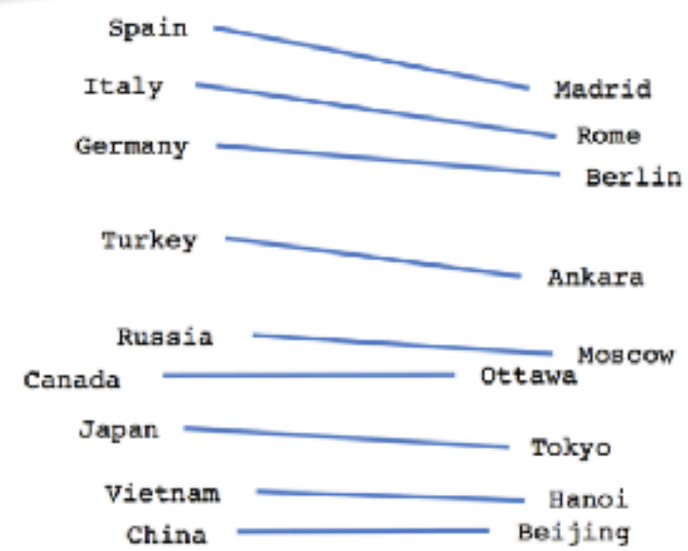


Male-Female



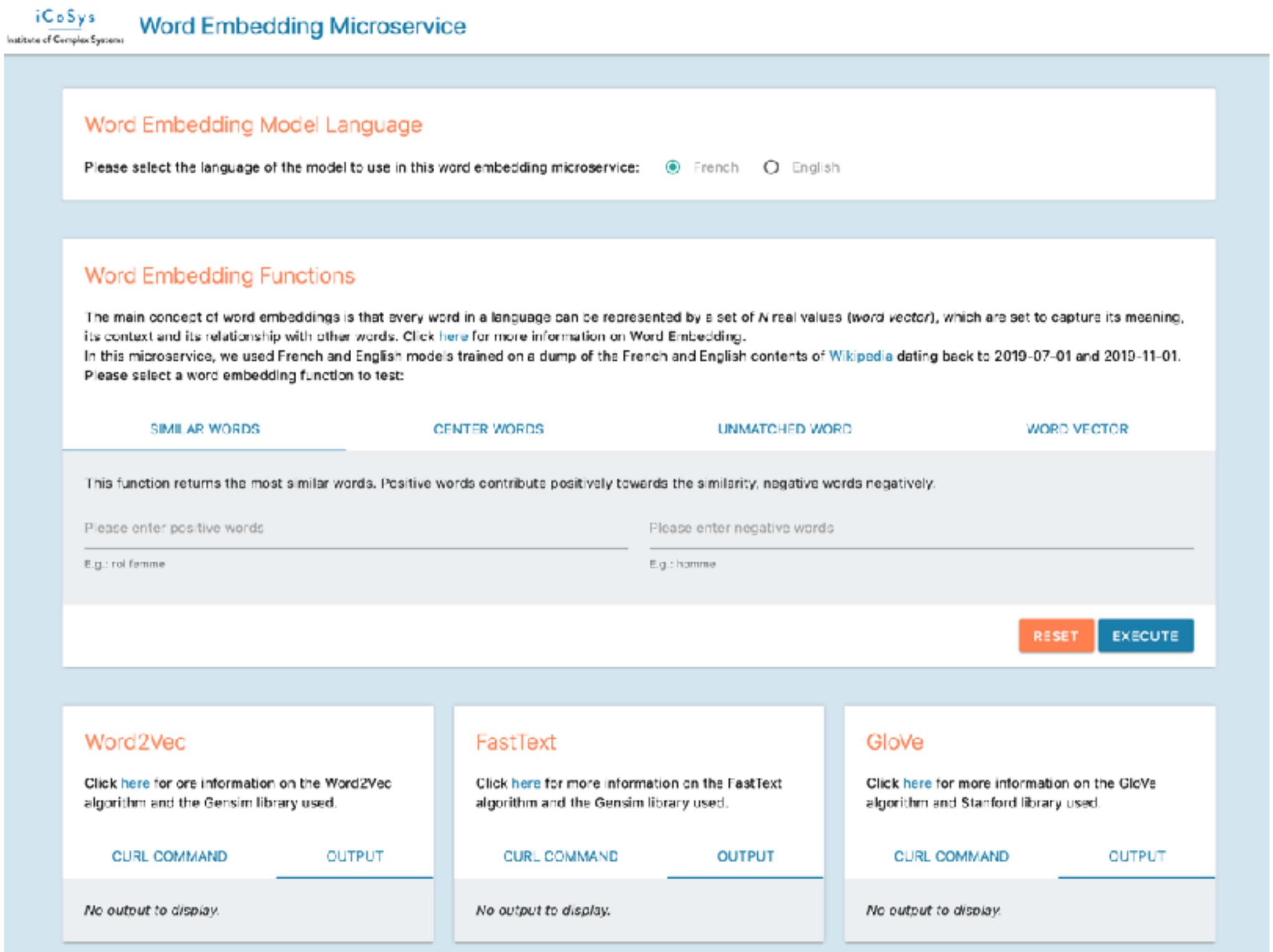
Verb tense

The path from a point to the other captures a semantic relationship: “capital of”



Country-Capital

Showcase - https://icoservices.kube.isc.heia-fr.ch/word-embedding/showcase



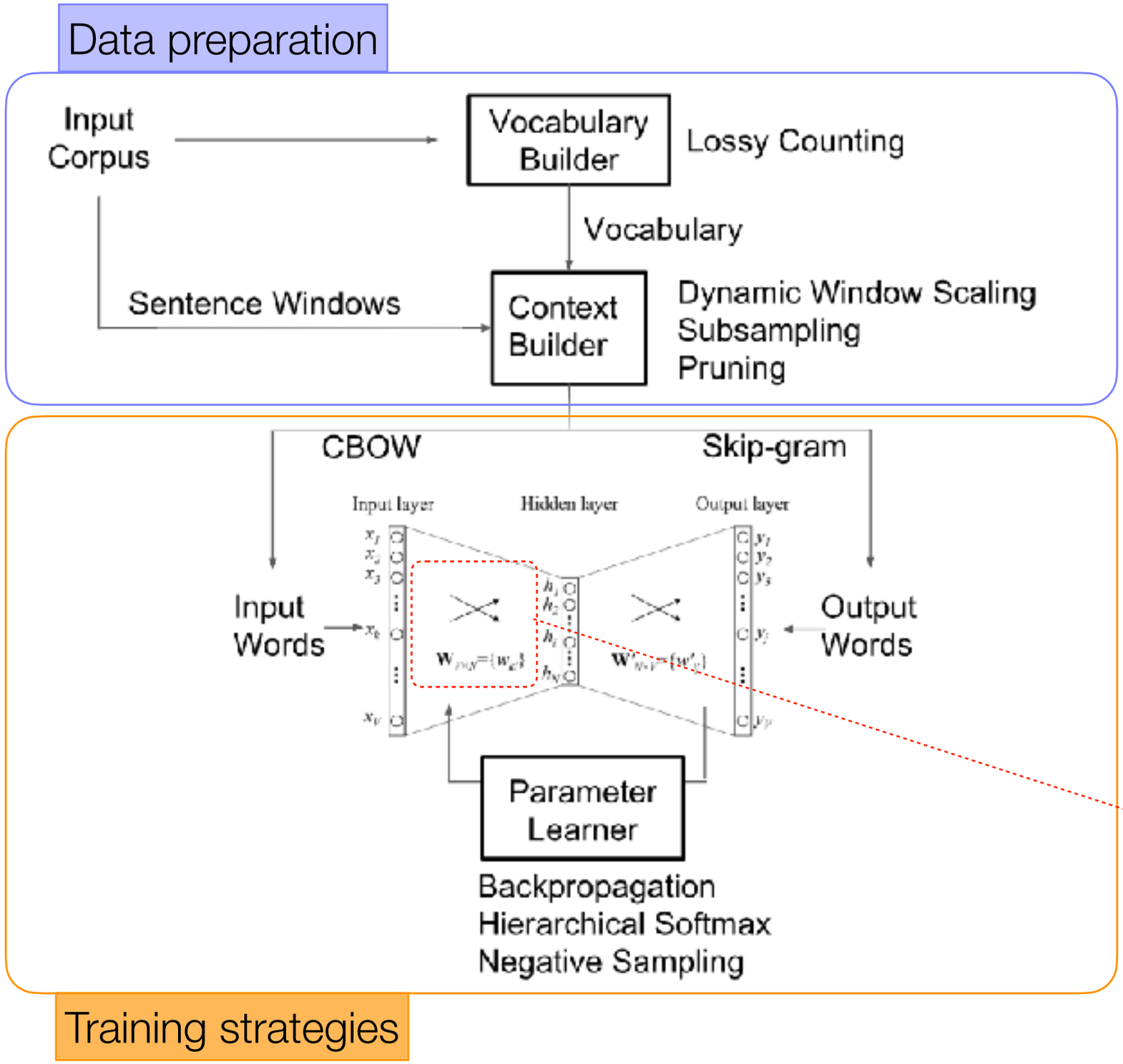
word2vec - applications

- Many machine learning and deep learning tasks use *word2vec* as a preliminary transformation of the word, i.e. as a feature extraction
- Once in the *vec* space, we can apply more efficiently classification, prediction, clustering, etc.
  - Named entity recognition
  - Document classification
  - Sentiment analysis
  - Paraphrase detection
  - Word clustering
  - Machine translation
  - …

3. Word2Vec

- Training principles
- Training strategies
  - One word context
  - Skip Gram
  - CBOW
- Practical considerations

word2vec - how did they train it ?



V vectors = final product

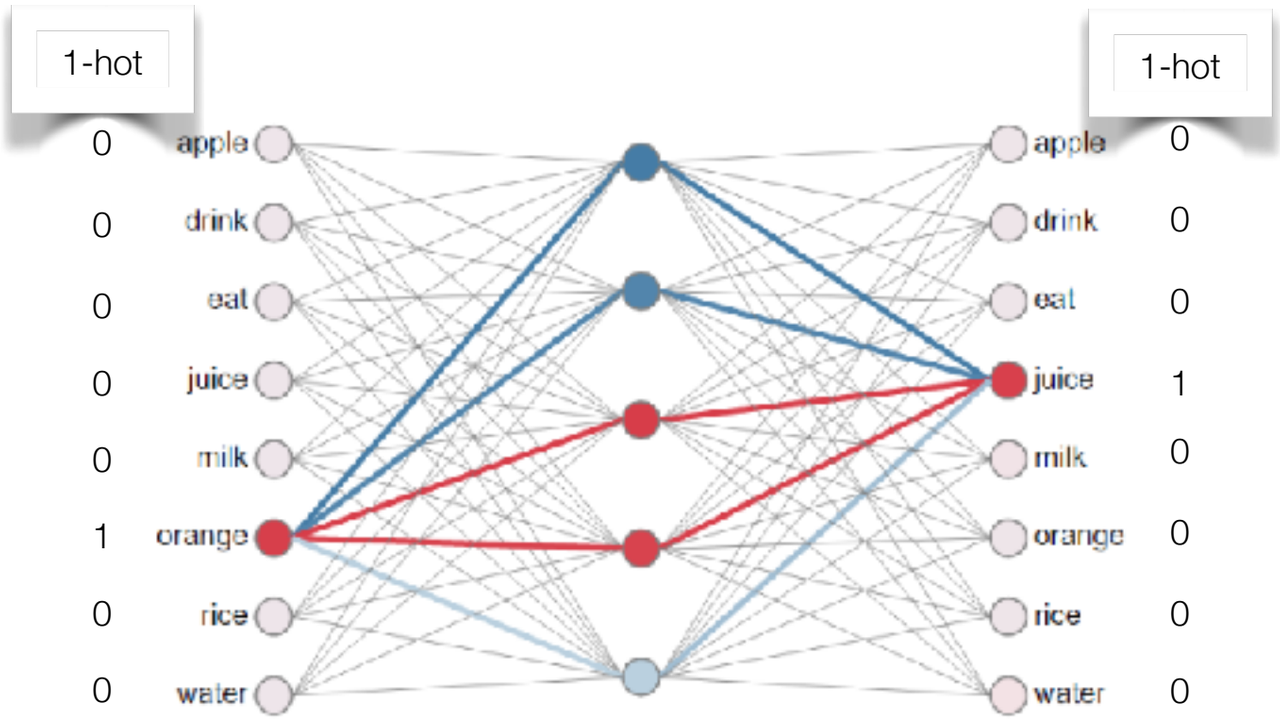
Figures taken from Xin Ring - School of Information - University of Michigan

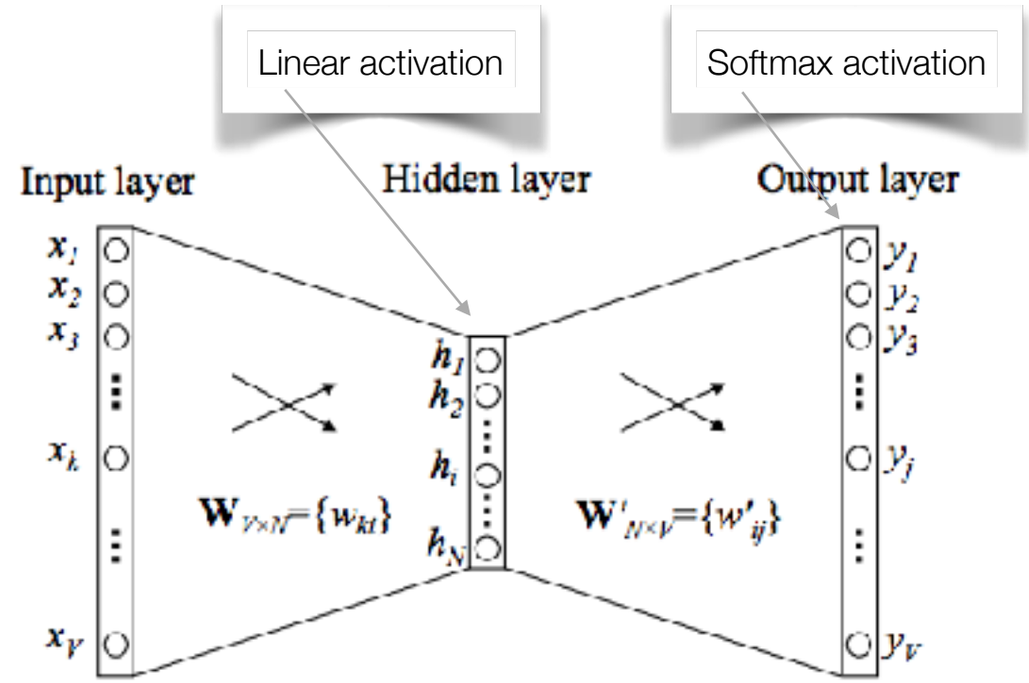
word2vec - training strategies

- One word context
  - Not really used, typically for educational purposes
- Skip-gram model
  - Kind of extension of one word context to multi word context at the output
- Continuous bag-of-word model - CBOW
  - Kind of extension of one word context to multi word context at the input
- Strategies are summarised in the next slides but if you want to dig further:
  - https://arxiv.org/pdf/1411.2738.pdf
  - https://www.youtube.com/watch?v=D-ekE-WIcds

One word context

- Neat online demo: https://ronxin.github.io/wevi/





As ‘orange’ is 1-hot, say in position 6 of vector x, h is then a simple lookup of the values in line 6 of the input weight matrix

$$\mathbf{h} = \mathbf{x}^T\mathbf{W} := \mathbf{v}_{w_I}$$

Embedding of word ‘orange’

Before the softmax, the output logit of, say ‘juice’, is the dot product of h by the 4th column vector of the output weight matrix

$$u_j = \mathbf{v}'_{w_j}{}^T \cdot \mathbf{h}$$

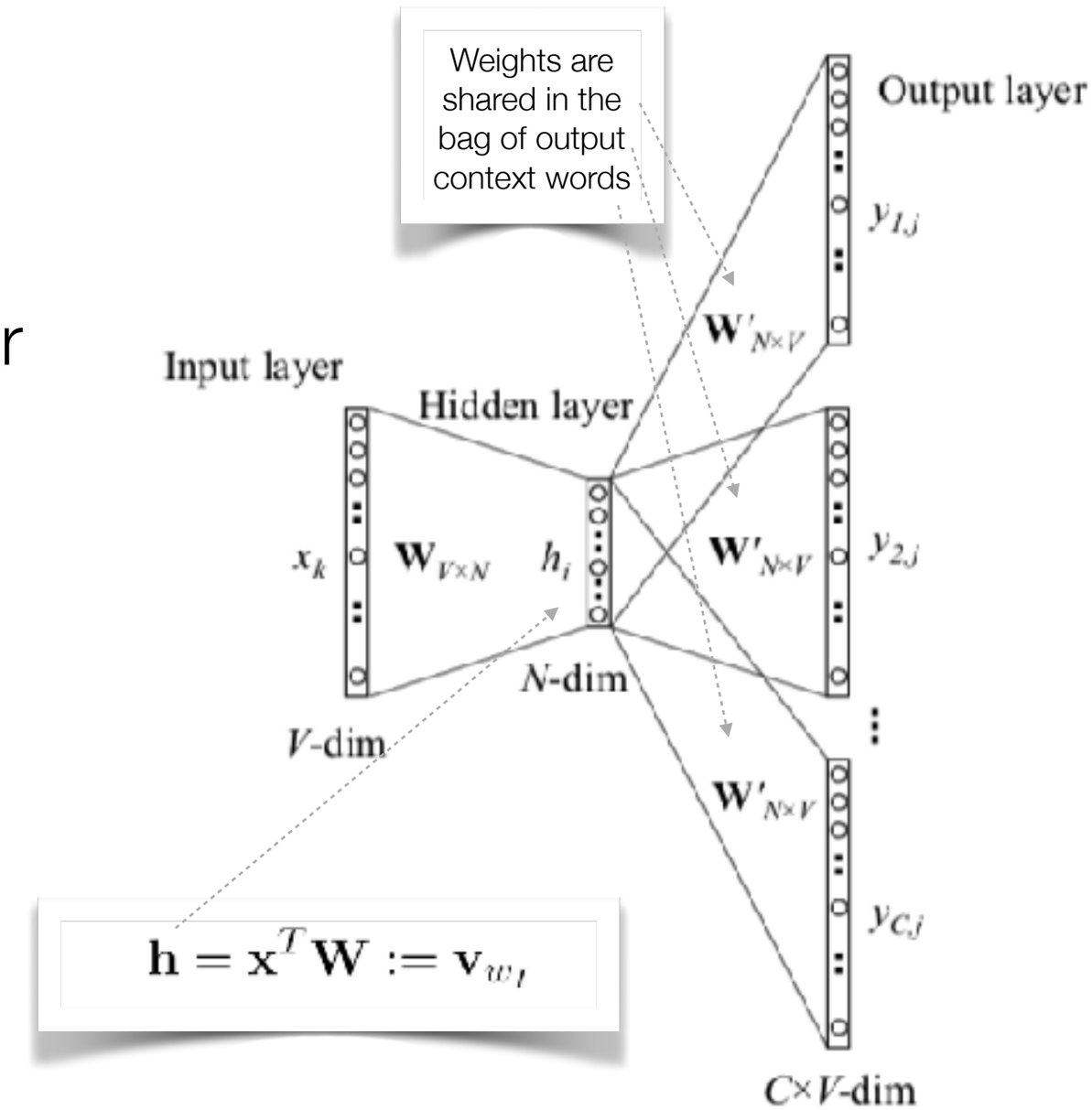
The softmax function can become problematic if we have large vocabulary. The denominator term can be cpu intensive.

$$p(w_j|w_I) = y_j = \frac{\exp(u_j)}{\sum_{j'=1}^{V}\exp(u_{j'})}$$

Figures taken from Xin Ring - School of Information - University of Michigan

Skip Gram model

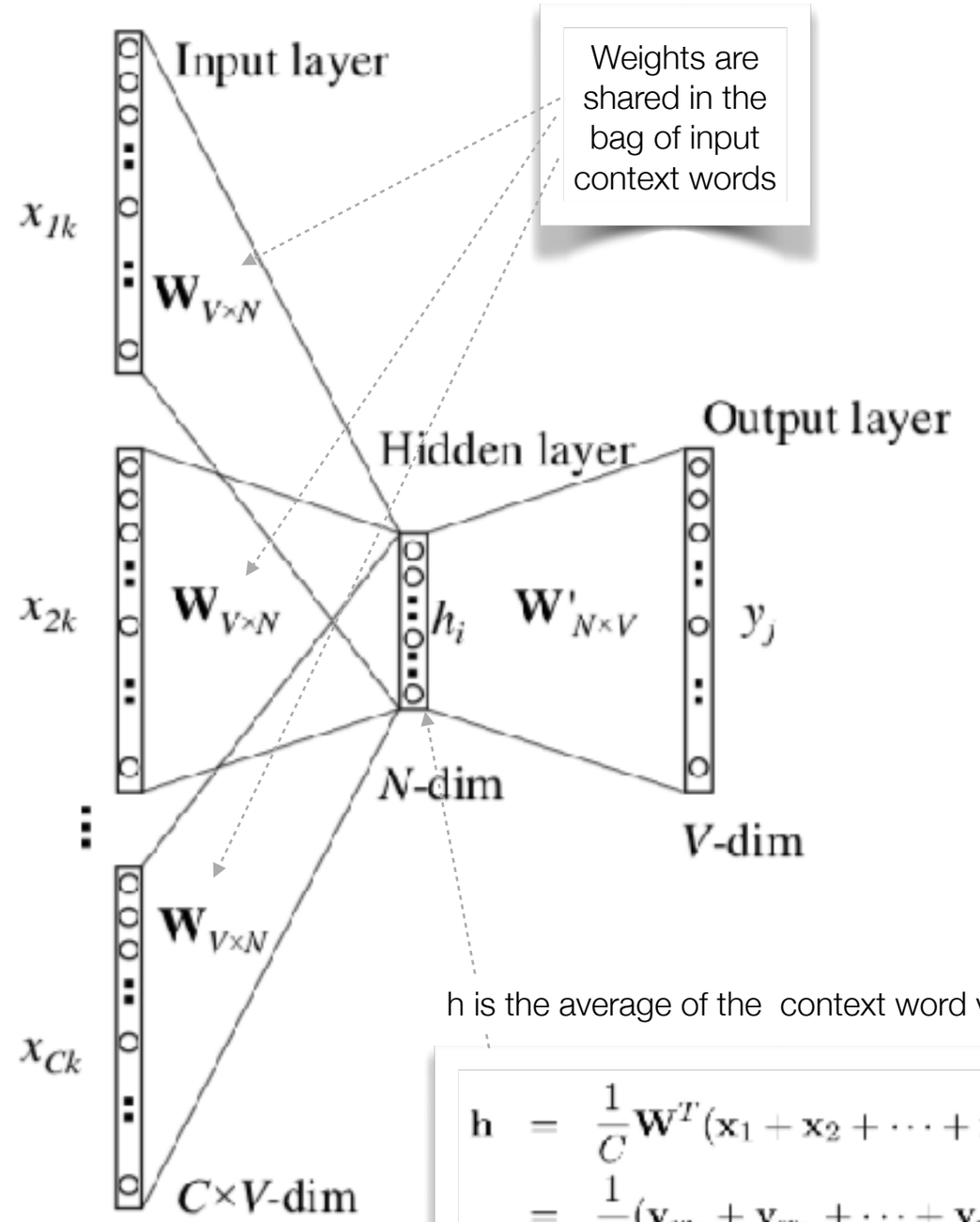
- Can be seen as an extension of one word context to multi word context at the output layer
- From a word, predict its bag of context words



Figures taken from Xin Ring - School of Information - University of Michigan

CBOW - Continuous Bag of Word

- Can be seen as an extension of one word context to multi-word context at the input layer
- From a bag of context words, predict a word



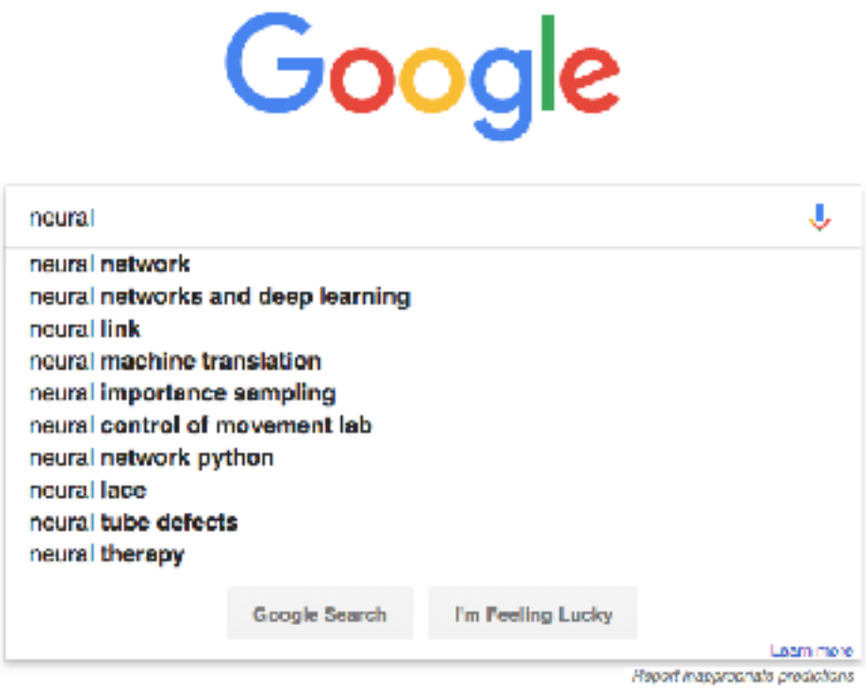
h is the average of the context word vectors

$$\begin{aligned}\mathbf{h} &= \frac{1}{C}\mathbf{W}^T(\mathbf{x}_1+\mathbf{x}_2+\cdots+\mathbf{x}_C)\\ &= \frac{1}{C}(\mathbf{v}_{w_1}+\mathbf{v}_{w_2}+\cdots+\mathbf{v}_{w_C})^T\end{aligned}$$

Figures taken from Xin Ring - School of Information - University of Michigan

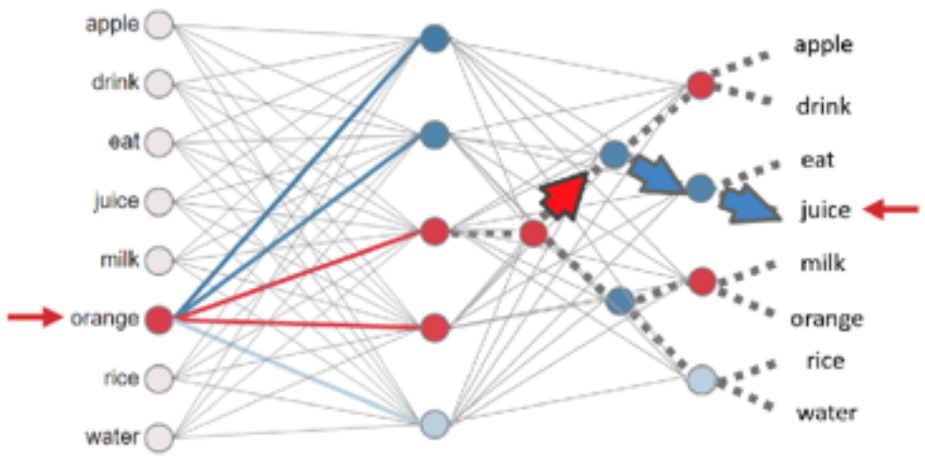
Activity

- Could we use word2vec for word prediction in the context of a search engine keyword completion?



word2vec - practical considerations on softmax

- The softmax and the derived update rules need to iterate through all the vocabulary
  - Impractical if V is large and if training corpora are large, which is the case most of the time
- 2 solutions
  - **Hierarchical Softmax** - Morin and Bengio, 2005
    - Build a binary tree on top of the output layer
    - https://www.youtube.com/watch?v=B95LTf2rVWM
  - **Negative sampling** - Mikolov et al, 2013
    - Update only a sample of the outputs, instead of all of them



word2vec - implementations

- A popular implementation is gensim
  - https://radimrehurek.com/gensim/models/word2vec.html
  - Pre-trained models and ready-to-download datasets: https://github.com/RaRe-Technologies/gensim-data

```
from gensim.models.word2vec import Word2Vec
import gensim.downloader as api

corpus = api.load('text8')  # download the corpus and return it opened as an iterable
model = Word2Vec(corpus)  # train a model from the corpus
model.most_similar("car")

"""
output:

[(u'driver', 0.8273754119873047),
 (u'motorcycle', 0.769528865814209),
 (u'cars', 0.7356342077255249),
 (u'truck', 0.7331641912460327),
 (u'taxi', 0.718338131904602),
 (u'vehicle', 0.7177008390426636),
 (u'racing', 0.6697118878364563),
 (u'automobile', 0.6657308340072632),
 (u'passenger', 0.6377975344657898),
 (u'glider', 0.6374964714050293)]

"""
```

Other embeddings

Glove

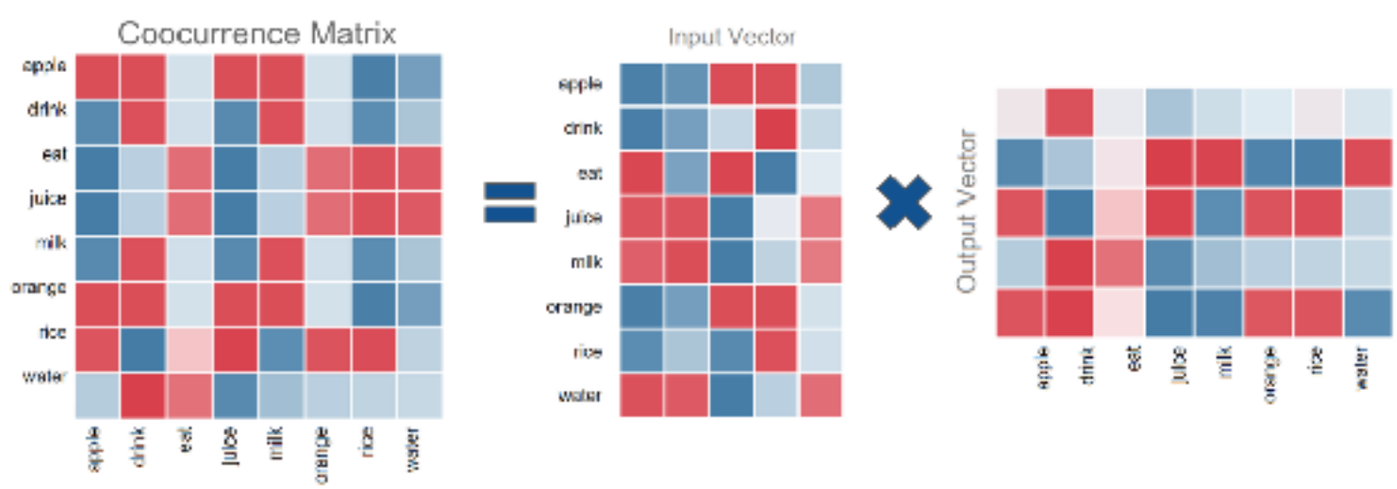
Fast text

ELMo

Other popular embeddings - GloVe

- GloVe - Global Vectors for Word Representation
  - https://nlp.stanford.edu/projects/glove/
  - Not based on pure ANN but on statistical approaches
    - dimensionality reduction of co-occurence matrix of 2 words in context
    - embeddings relate to the probabilities that two words appear together.
    - *word2vec* and approaches based on co-occurence matrices are actually related

O. Levy and Y. Goldberg, “Neural Word Embedding as Implicit Matrix Factorization”, NIPS 2014



Figures taken from Xin Ring - School of Information - University of Michigan

Other popular embeddings - FastText

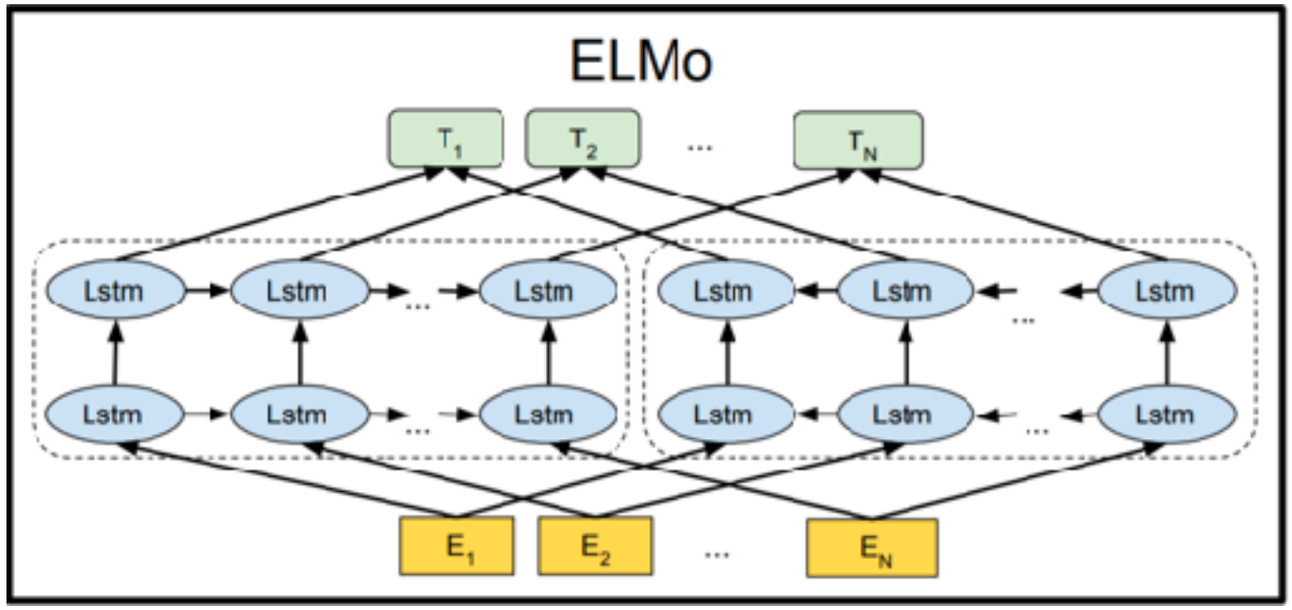
- Extension to Word2Vec proposed by Facebook in 2016
  - https://arxiv.org/pdf/1607.04606.pdf
  - https://radimrehurek.com/gensim/models/fasttext.html
- Instead of feeding individual words into the NN, FastText breaks words into several *n*-grams (i.e. sub-words)
  - E.g. the tri-grams for the word “apple” is “app”, “ppl”, and “ple”
- The word embedding vector for a word will be the sum of all of its *n*-grams
- After training the NN, we’ll have word embeddings for all the *n*-grams of the training set
- Main advantage: **out-of-vocabulary words can be represented** since their *n*-grams probably appears in other words of the training set
  - Useful for company names: “creativeinfotech” could be close to IT concepts in the embedding space
  - Useful for languages using compound words

https://www.dartmouth.edu/~deutsch/Grammatik/Wortbildung/Komposita.html

Figures taken from Xin Ring - School of Information - University of Michigan

Towards more complex embeddings - ELMo

- Intuition: meaning changes with context
  - "The *plane* took off at exactly nine o'clock."
  - "The *plane* surface is a must for any cricket pitch."
  - "*Plane* geometry is fun to study."
- The word *plane* should have different embeddings according to its context
- Instead of using a fixed embedding for each word, like word2vec do, ELMo looks at the entire sentence before assigning each word in its embedding
- The term "word embedding" leaves the place to "language models".
- See:
  - 2018, https://arxiv.org/abs/1802.05365
  - https://medium.com/saarthi-ai/elmo-for-contextual-word-embedding-for-text-classification-24c9693b0045



Wrap-up

- **Word Embedding**
  - Projection of a word into vectors of real numbers
  - 3 strategies to use word embedding:
    - Train from scratch the embedding layer on a given data set and for a given task
    - Use a pre-trained “generic” embedding such as word2vec or GloVe and freeze the layer - similar to transfer learning concept
    - Do both: init the embedding layer with pre-trained weights and fine tune it through training.
  - **Word2Vec** allows to preserve contextual information in the embedding space
    - Similar approach is **GloVe**
  - **FastText** is similar to word2vec but is able to handle out-of-vocabulary words
  - **ELMo** takes the whole context of a sentence before doing the embedding
    - ELMo and Transformer architectures (see next week) were foundation ideas for recent breakthrough-ing approaches for language models, including BERT